SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

StockMaster AI

Team: Galaxy Annihilator

Introduction

StockMaster AI is an intelligent, AI-driven inventory management and autonomous procurement system developed for milk tea shop operators in Malaysia. This document presents the strategic technical solution addressing the problem statement identified in the Product Requirement Documentation — namely, the inefficiency of manual inventory tracking, stockout incidents, and sub-optimal supplier selection in small F&B businesses.

Purpose

This System Analysis Document highlights the technical scope and design-related decisions behind the StockMaster AI development. It serves as the authoritative reference for developers, testers, and evaluators to understand the overall high-level architecture, data flows, AI integration strategy, and system boundaries of this project.

The key elements of the system that have been covered are as follows:

Architecture: A client-server architecture with a React (Create React App) frontend communicating with a Python Flask REST API backend. The backend exposes 25+ tool endpoints that the AI agent orchestrates via OpenAI-compatible function calling to the ILMU.ai GLM-5.1 model.

Data Flows: How inventory data flows from JSON file storage through the Flask API, gets processed by the AI agent's tool execution layer, and returns structured responses to the React dashboard for user display.

Model Process: The end-to-end workflow of core features: real-time stock monitoring, AI-powered gap detection, multi-supplier price comparison with unit normalization, and automated order draft generation.

Role of Reference: This document serves as a reference for all developers and testers to understand the high-level architecture, API contracts, and design rationale of StockMaster AI.

Background

Malaysian milk tea shops manage 20-30+ ingredients across 9 categories (Dairy, Boba, Tea, Sweeteners, Packaging, Toppings, Janitorial, Equipment, Supplies) sourced from multiple competing suppliers. The existing manual processes — paper records, WhatsApp messages, and basic spreadsheets — lead to frequent stockouts, over-ordering, and inability to optimize procurement costs across 22+ supplier options.

Previous Version: No previous version of this system existed. Shop operators previously relied entirely on manual methods. The StockMaster AI project was developed from scratch for UMHackathon 2026.

Changes in Major Architectural Components: The system introduces a novel agentic AI architecture where the LLM (GLM-5.1) acts as an autonomous decision-making layer that orchestrates 25+ backend tools via function calling — a fundamentally different approach from traditional CRUD inventory applications.

Target Stakeholders

Stakeholders	Roles	Expectations
Milk Tea Shop Owner/Operator	Browse inventory status, receive stock alerts, trigger restocking actions, interact with AI assistant using natural language (including Manglish)	Simple, intuitive dashboard. AI understands local language. Automated supplier price comparison and order drafting.
Shop Manager	Monitor stock levels daily, manage supplier relationships, track supplier debts, export inventory reports	Accurate real-time stock data. Easy CSV export. Supplier debt visibility.
Supplier	Receive purchase orders via WhatsApp or email with correct item details, quantities, and pricing	Clear, formatted order messages. Accurate pricing and quantities.
Development Team	Build and maintain the React frontend, Flask backend, and AI agent integration	Clear API contracts (25+ documented endpoints), modular codebase, well-documented tool schemas.
QA Team	Validate system behavior for inventory CRUD operations, AI responses, and edge cases	Defined API request/response formats, test scripts (test_functions.py, test.bat), and coverage of edge cases.

System Architecture & Design

High Level Architecture

Overview:

Type	Details
System	Web Application (React SPA) + REST API Backend (Python Flask)
Architecture	Client-Server with Agentic AI Orchestration Layer
AI Engine	ILMU.ai GLM-5.1 via OpenAI-compatible API (https://api.ilmu.ai/v1)
Data Storage	JSON file-based (inventory.json, supplier_debt.json, price_history.json) with Firebase Firestore configured for production migration

StockMaster AI is structured as a client-server system with three distinct layers: (1) React Frontend Dashboard — a single-page application served on port 3000 that provides the visual inventory management interface with multi-language and multi-currency support; (2) Flask Tools API — a Python REST API on port 5001 that exposes 25+ endpoints for inventory CRUD, supplier queries, price comparison, order generation, and reporting; (3) AI Agent Layer — a Python module (agent.py) that connects to ILMU.ai's GLM-5.1 model and autonomously orchestrates the Tools API endpoints via function calling to fulfill natural language user requests.

LLM as Service Layer

The architecture integrates GLM-5.1 as an autonomous service layer, not merely a generic AI module. The integration works as follows:

- Prompt Construction: The agent builds messages with a comprehensive system prompt (~2000 tokens) defining the AI's role as 'StockMaster AI, an intelligent inventory management assistant for a milk tea shop in Malaysia.' This includes Manglish expression mappings (e.g., 'habis' = finished, 'kosong' = empty).
- Context Window Contents: System prompt + user message + conversation history + tool call results from previous iterations. Inventory and supplier data are injected via tool calls (get_all_inventory, get_suppliers) rather than statically, keeping the initial context lean.
- Response Parsing: The agent checks if the LLM response contains tool_calls. If yes, it extracts the function name and arguments, executes the corresponding HTTP request to the Tools API, formats the result, and feeds it back as a tool message. If no tool_calls, the response is treated as the final answer.
- Token Enforcement: Tool results are formatted and truncated (max 2000 chars for generic results, max 30 items for inventory lists). Maximum 10 iterations per agent run to prevent runaway inference costs.
- Major API Calls: React Frontend -> Flask API (port 5001) for all tool endpoints. Agent.py -> ILMU.ai API (https://api.ilmu.ai/v1) for LLM inference. Agent.py -> Flask API (port 5001) for tool execution via HTTP requests.

Sequence Diagram — AI-Assisted Restock Flow

The following describes the step-by-step interaction for a user requesting stock gap analysis and automated ordering:

1. User types 'susu habis, nak order' in the AI Assistant chat interface

2. React Frontend sends POST to /api/chat with user message

3. Agent.py constructs messages array with system prompt + user message

4. Agent.py calls ILMU.ai GLM-5.1 API with 25 tool schemas

5. GLM returns tool_call: get_low_stock_items()

6. Agent.py executes GET http://localhost:5001/api/tools/get_low_stock_items

7. Flask API reads inventory.json, filters items where currentStock < minStock, returns JSON

8. Agent.py formats result and feeds back to GLM as tool message

9. GLM returns tool_call: get_optimal_supplier(name='Fresh Whole Milk')

10. Agent.py executes GET http://localhost:5001/api/tools/get_optimal_supplier?name=Fresh+Whole+Milk

11. Flask API compares 3 suppliers (Budget Dairy RM68, Dairy Farms RM73.12, Premium Dairy RM85), normalizes prices per unit, returns best option

12. Agent.py formats result and feeds back to GLM

13. GLM returns tool_call: generate_order_draft(supplier='Budget Dairy Supply', items=[...], format='whatsapp')

14. Agent.py executes POST to Flask API, receives formatted WhatsApp message

15. GLM provides final response summarizing actions taken with order details

16. React Frontend displays AI response to user

Technological Stack

Layer	Technology	Justification
Frontend	React 19.2.5 (Create React App)	Rapid prototyping with hot-reload. Component-based architecture for dashboard, tables, forms. Built-in test runner.
Backend API	Python Flask 2.3+ with Flask-CORS	Lightweight REST API framework. Easy to define 25+ endpoints. CORS enabled for cross-origin React requests.
AI/LLM	ILMU.ai GLM-5.1 via OpenAI SDK	OpenAI-compatible API enables use of standard openai Python package. Strong function-calling support for 25+ tool schemas. Multilingual (EN/MS/ZH/TA).
Database (Dev)	JSON files (inventory.json, supplier_debt.json, price_history.json)	Simple, hackathon-friendly. No database setup required. Human-readable data format.
Database (Prod)	Firebase Firestore (configured in firebase.js)	Cloud-native NoSQL. Collections pre-defined: suppliers, inventory, orders, gaps. Real-time listeners via onSnapshot.
Testing	PyTest (backend), Jest/React Testing Library (frontend), test.bat (integration)	Automated test functions for 8 API endpoints. Batch script for quick health verification.

Key Data Flows

Data Flow Description

The data movement across the StockMaster AI system involves the following entities and processes:

- External Entities: Shop Operator (user), Supplier (receives orders via WhatsApp/email), ILMU.ai API (LLM inference)
- Processes: Inventory CRUD (25 tool endpoints), AI Agent Orchestration (agent.py with 10-iteration loop), Price Normalization Engine (unit conversion across liters/kg/units), Order Draft Generation (WhatsApp/email format)
- Data Stores: inventory.json (25 items with id, name, category, currentStock, minStock, unit), supplier data embedded in inventory.json (22 suppliers with items, units, priceRM), supplier_debt.json (debt tracking per supplier), price_history.json (timestamped price fluctuations)

Database Schema

Inventory Item Schema:

- id (string) — Unique identifier
- name (string) — Item name (e.g., 'Fresh Whole Milk')
- category (string) — One of: Dairy, Boba, Tea, Sweeteners, Packaging, Toppings, Janitorial, Equipment, Supplies
- currentStock (number) — Current stock level
- minStock (number) — Minimum threshold for low-stock alert
- unit (string) — Unit of measurement (e.g., 'Case (4 x 1 Gallon)', '3 kg bag')

Supplier Schema (embedded in inventory.json):

- name (string) — Supplier company name
- items (array) — List of items with: name, unit, priceRM

Supplier Debt Schema:

- total_debt_rm (number) — Sum of all outstanding debts
- debts (array) — Per-supplier: supplier_name, amount_owed

Functional Requirements & Scope

Minimum Viable Product

#	Feature	Description
1	Real-Time Inventory Dashboard	Web dashboard displaying 25 items across 9 categories with stock levels, status indicators (In Stock/Low/Critical), supplier info. Multi-language (EN/MS/ZH/TA) and multi-currency (7 currencies) support.
2	AI-Powered Stock Gap Analysis	Automated detection of items below minimum thresholds. Priority classification (CRITICAL/HIGH/MEDIUM). Estimated restocking costs calculated from supplier prices.
3	Supplier Price Comparison & Optimization	Unit-price normalization engine comparing prices across 22 suppliers with different package sizes. Identifies optimal supplier and calculates savings.
4	Automated Order Drafting	One-click order placement with auto-supplier selection. WhatsApp and email order message generation addressed to specific suppliers.
5	Natural Language AI Assistant	Conversational AI interface understanding English, Malay, Manglish, Mandarin, Tamil. Agentic tool orchestration with up to 10 iterations per query.

Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	System must handle concurrent inventory queries without data corruption	JSON file storage is single-threaded; Firebase Firestore migration path is configured for production scale.
Reliability	Inventory operations should have consistent read/write behavior. Failed API calls should not corrupt data.	Flask API uses atomic file read/write operations. Error handlers for 404 and 500 responses return structured JSON.
Maintainability	Each component (frontend, tools API, AI agent) is separated and independently deployable.	React app on port 3000, Flask API on port 5001, Agent module is standalone Python. Independent startup scripts.
Token Latency	AI agent responses should complete within 30 seconds under normal conditions.	Max 10 iterations with 30s timeout per API call. Tool results truncated to 2000 chars to minimize token usage per iteration.
Cost Efficiency	Average token cost per session should not exceed RM0.10 equivalent.	System prompt is fixed-length (~2000 tokens). Tool results are formatted concisely. Max 10 iterations caps total tokens at ~8000 per session.

Out of Scope / Future Enhancements

- User authentication, login system, and role-based access control
- Real payment gateway integration (Stripe, FPX, etc.)
- Vision model integration for fridge/storage photo analysis (glm-4.5v planned but not implemented)
- Multi-branch outlet management with separate inventory databases
- Mobile native application (iOS/Android)
- Predictive demand forecasting using historical sales data

Monitor, Evaluation, Assumptions & Dependencies

Technical Evaluation

Grayscale Rollout & A/B Testing: For the hackathon demo, the system is deployed locally with demo data pre-loaded. In a production scenario, new features (e.g., new AI tool endpoints) would be released behind feature flags, monitored for error rates, and then gradually enabled for all users.

Emergency Rollback (ER): The JSON-based data storage allows quick rollback by restoring previous inventory.json from version control. The Flask API has no database migrations — redeployment with previous code is sufficient for full rollback.

Priority Matrix

- P1 Critical: If ILMU.ai API returns errors for >1 minute, fallback to frontend's built-in pattern-matching for basic commands.
- P2 High: If inventory.json becomes corrupted, restore from Git repository's last known good state.
- P3 Medium: If price normalization produces incorrect results for a specific unit format, add the format to the regex parser.

Monitoring

The Flask API includes a health_check endpoint (GET /api/tools/health_check) that returns server status, timestamp, and port information. The agent.py module logs all tool calls, arguments, and results to stdout with iteration numbers for debugging. The test.bat script provides a quick 5-point health verification suite.

Assumptions

- Users have stable internet connections for ILMU.ai API calls.
- Shop operators have access to a modern web browser (Chrome, Firefox, Edge).
- Supplier prices in the database are reasonably current (updated manually or via price tracking).
- The ILMU.ai API key has sufficient quota for the expected number of user sessions.

External Dependencies

Tools	Purpose	Risks
ILMU.ai GLM-5.1 API	Core LLM reasoning engine — handles prompt inference, multi-step agentic tool orchestration, and generating structured natural language output	HIGH — If GLM API is rate-limited or unavailable, core AI features fail. Mitigation: Frontend built-in fallback handles basic commands without LLM.
Firebase Firestore	Production database for persistent inventory, supplier, and order data with real-time sync	MEDIUM — Currently using JSON files for hackathon. Migration to Firestore needed for production multi-user support.
WhatsApp (wa.me links)	Order delivery channel — generates pre-filled WhatsApp messages for supplier communication	LOW — Uses standard wa.me URL protocol. No API dependency. Works on any device with WhatsApp installed.

Project Management & Team Contributions

The development followed an agile sprint approach within the UMHackathon 2026 preliminary round timeframe (~10 days).

Project Timeline: Days 1-2: Project planning, architecture design, and supplier data collection. Days 3-5: Backend development (Flask Tools API with 25+ endpoints, inventory.json schema design). Days 6-8: Frontend development (React dashboard, multi-language/currency support, AI chat interface). Days 9-10: AI agent integration (GLM-5.1 function calling, system prompt engineering), testing, and documentation.

Recommendations: For production deployment: (1) Migrate from JSON files to Firebase Firestore for concurrent multi-user support. (2) Implement Redis caching for supplier price data to reduce redundant file reads. (3) Add rate limiting on AI agent calls to control ILMU.ai API costs. (4) Implement the vision model (glm-4.5v) for fridge photo analysis as planned in the implementation roadmap.